

Promises

Asynchronität als Basis

```
// reading a file synchronously const fs = require('fs');  
const data = fs.readFileSync('/file.md');  
    // blocks here until file is read
```

Problem

- JavaScript ist single-threaded
- Skripte, die blockierenden IO verwenden sind nicht performant!
- Die meisten JavaScript-Standard-Funktionen erwarten daher **Callbacks**, die das Ergebnis verarbeiten (**kein return Wert**):

```
// reading a file asynchronously  
const fs = require('fs');  
fs.readFile('/file.md', function(err, data){  
    if (err) throw err;  
});
```

Schlecht lesbarer Code, Programmablauf unklar!

**Anything that needs to happen after a callback
has fired needs to be invoked from within it**

Promises

Ausweg aus der Callback-Hell

Der Programmablauf wird durch ineinander verschachtelte Callbacks geprägt

```
chooseToppings(function(toppings) {  
  placeOrder(toppings, function(order) {  
    collectOrder(order, function(pizza) {  
      eatPizza(pizza);  
    }, failureCallback);  
  }, failureCallback);  
}, failureCallback);
```

Promises strukturieren diesen konditionalen Ablauf besser. Noch einmal verbessert wird die Lesbarkeit durch Verwenden der then-Notation (statt resolve)

```
chooseToppings()  
  .then(toppings => placeOrder(toppings))  
  .then(order => collectOrder(order))  
  .then(pizza => eatPizza(pizza))  
  .catch(failureCallback);
```

<https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Asynchronous/Promises>

Promises

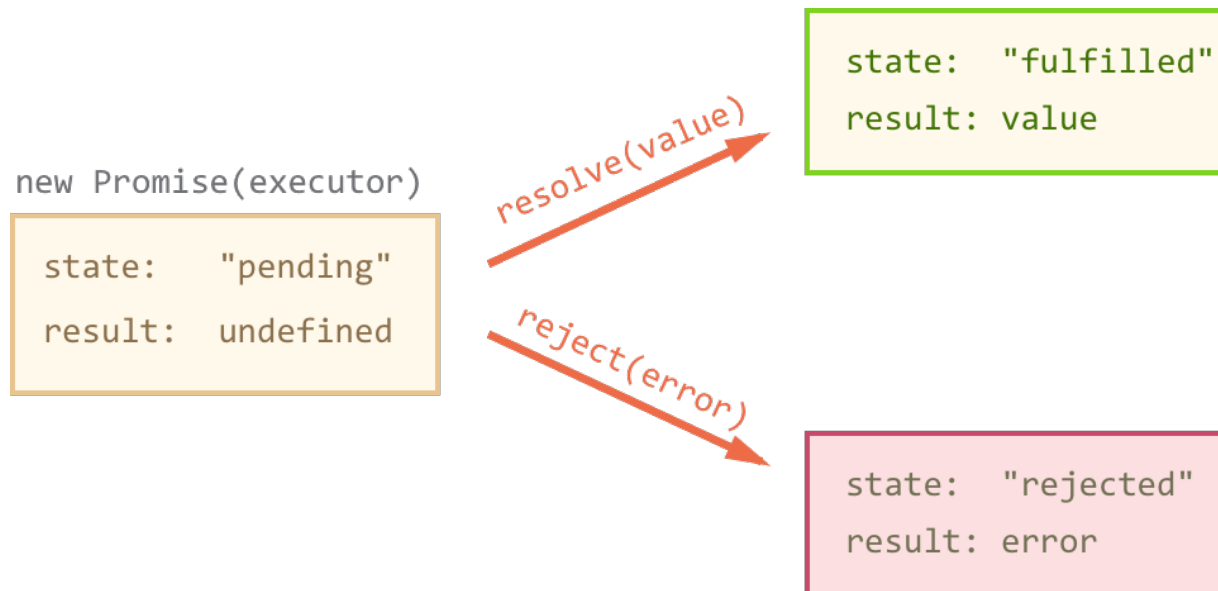
Stellvertreter/Proxys für ein zukünftiges Resultat

- Das **Promise**-Interface repräsentiert einen **Platzhalter** (ähnlich zu futures in Java)
- Es gibt **drei Zustände** für den Platzhalter:
Pending (Startzustand), **Fulfilled** und **Rejected**
- An die Zustandsübergänge werden im Promise zwei Funktionen (faktisch callbacks) geheftet, die über die Event-Loop ausgeführt werden, wenn sie zutreffen und laut Queue dran sind
 - `resolve(value)`: erfolgreicher Ausgang - Platzhalter wird mit Wert gefüllt
 - `reject(reason)`: Misserfolg - Platzhalter wird mit Fehlermeldung gefüllt
- Erzeugt wird ein Promise mit einem `new`. Hierbei wird ein **Executor** übergeben, der als Parameter die beiden genannten Callbacks hat.
- Der **Executor wird direkt synchron ausgeführt**, wobei `resolve` und `reject` je nach Verwendung direkt oder über die Event Queue später aufgerufen werden. Diese Aufrufe **hinterlegen** dann die Handler-Ausführung auf die **Microtask Queue** (existiert zusätzlich zur Event Queue) ein. Die Handler laufen dadurch **später**, nachdem der aktuelle Call Stack fertig ist und werden durch den Zustandsübergang getriggert.
- Die **Resultate können auch durch `then`**, bzw. **`catch`** abgefragt werden, was so zu einer Entflechtung der Callback-Verschachtelung führt

Syntax

Asynchronität durch Promises

3 Zustände:



<https://javascript.info/promise-basics>

```
new Promise(executor);
```

```
new Promise(function(resolve, reject) { ... });
```

Syntax

Asynchronität durch Promises

```
1 let promise = new Promise(function(resolve, reject) {  
2   // the function is executed automatically when the promise is constructed  
3  
4   // after 1 second signal that the job is done with the result "done"  
5   setTimeout(() => resolve("done"), 1000);  
6 });
```

Executor als anonyme Funktion
wird direkt ausgeführt

Später Aufruf von resolve()
über die Event Queue

<https://javascript.info/promise-basics>

`new Promise(executor);`

`new Promise(function(resolve, reject) { ... });`

- **executor ruft direkt oder indirekt** resolve() (alles OK) und reject() (nicht OK). Damit müssen auslösende Ereignisse hier stehen
- Resolve und/oder reject müssen entweder explizit aufgerufen werden oder indirekt (thenable)
- Hinweis: Der Return-Wert kann wieder ein Promise sein! Daher ist Chaining möglich. Ein resolve() liefert also wieder ein Promise



<https://javascript.info/promise-basics>

Promise

Nutzungsbeispiel und Abarbeitungsfolge

```
const promise1 = new Promise( executor: (resolve, reject) =>{  
  console.log("Los geht es")  
  // Promises ermöglichen eine elegantere Verarbeitung  
  // asynchroner Ereignisse. Hier ein Timeout  
  setTimeout( handler: () => resolve( value: 'Sieht aus wie mitten drin'), timeout: 0)  
})  
console.log("Zweiter")  
promise1.then((value) => console.log(value) )  
console.log("Man meint ich käme zum Schluss")
```

Gerne auch mit Arrow-
Funktion formulieren

Alles was hier steht wird im aktuellen Call-
Stack ausgeführt. Der Trick liegt darin, in den
Callbacks asynchroner Aufrufe das resolve
zu machen, also wenn etwas relevantes
passiert ist.
Hier kommen WebAPI und libuv ins Spiel

Then erwartet eine Funktion, hier durch arrow-
Notation

Los geht es

Zweiter

Man meint ich käme zum Schluss

Sieht aus wie mitten drin

Process finished with exit code 0

Promise

Nutzungsbeispiel und Abarbeitungsfolge

```
const promise1 = new Promise( executor: (resolve, reject) =>{  
    console.log("Los geht es")  
} // Promises ermöglichen eine elegantere Verarbeitung  
// asynchroner Ereignisse. Hier ein Timeout  
setTimeout( handler: () => resolve( value: 'Sieht aus wie mitten drin'), timeout: 3000)  
})  
console.log("Zweiter")  
promise1.then(console.log)  
console.log("Man meint ich käme zum Schluss")
```

⚠ 1

Console.log ist bereits eine Funktion. Arrow-Funktion wird nur benötigt, wenn man etwas zusätzliches machen will:
`promise.then(v => { console.log("Wert:", v); doSomething(v); });`

Los geht es

Zweiter

Man meint ich käme zum Schluss

Sieht aus wie mitten drin

Process finished with exit code 0

Nutzungsbeispiel und Abarbeitungsfolge



Promise

Nutzungsbeispiel und Abarbeitungsfolge

```
const promise1 = new Promise( executor: (resolve, reject) =>{  
  console.log("Los geht es")  
  // Promises ermöglichen eine elegantere Verarbeitung  
  // asynchroner Ereignisse. Hier ein Timeout  
  setTimeout( handler: () => resolve( value: 'Sieht aus wie mitten drin'), timeout: 0)  
})  
console.log("Zweiter")  
//promise1.then(console.log) ←  
console.log("Man meint ich käme zum Schluss")
```

Promise wird zwar erfüllt, der Wert aber nie durch then abgegriffen.
Resolve liefert nur einen Wert.

Los geht es

Zweiter

Man meint ich käme zum Schluss

Process finished with exit code 0

Syntax

Zugriff auf das Resultat mittels then/catch

```
1 let promise = new Promise(resolve => {
2   setTimeout(() => resolve("done!"), 1000);
3 });
4
5 promise.then(alert); // shows "done!" after 1 second
```

Ohne then() wird der von resolve gelieferte Wert nie verarbeitet. Anschaulich wird durch das then() ein Chaining betrieben. Then ruft den Handler alert mit dem Argument (value) auf!

```
1 let promise = new Promise((resolve, reject) => {
2   setTimeout(() => reject(new Error("Whoops!")), 1000);
3 });
4
5 // .catch(f) is the same as promise.then(null, f)
6 promise.catch(alert); // shows "Error: Whoops!" after 1 second
```

<https://javascript.info/promise-basics>

```
1 let promise = new Promise(function(resolve, reject) {
2   setTimeout(() => resolve("done!"), 1000);
3 });
4
5 // resolve runs the first function in .then
6 promise.then(
7   result => alert(result), // shows "done!" after 1 second
8   error => alert(error) // doesn't run
9 );
```

```
1 let promise = new Promise(function(resolve, reject) {
2   setTimeout(() => reject(new Error("Whoops!")), 1000);
3 });
4
5 // reject runs the second function in .then
6 promise.then(
7   result => alert(result), // doesn't run
8   error => alert(error) // shows "Error: Whoops!" after 1 second
9 );
```

Was ist das Ergebnis auf der Konsole?

```
console.log( 'a' )

const p = new Promise( executor: function ( resolve, reject ) {
  console.log( 'b' )
  setTimeout( handler: () => {
    console.log('D')
    resolve ( value: 'E' )
  }, timeout: 0 );
} );

// Other synchronous stuff, that possibly takes a very long time to process
p.then(y => console.log(y))
console.log( 'c' )
```

a
b
c
D
E

Syntax

from Callback to Promise - Anwendung

fsreadfile mit Callback:

```
// reading a file asynchronously
const fs = require('fs');
fs.readFile(filePath, function(err, data){
    if (err) throw err;
    console.log('CONTENT:', data);
});
```

Anwendung von Promises am Beispiel von fsreadfile:

```
readFilePromisified(filePath, {encoding: 'utf8'})
.then(data => console.log('CONTENT:', data))
.catch(err => console.log('ERROR:', err))
```

Promises

Die Implementierung ist zum aktuellen Stand nicht ganz leicht

```
const fs = require('fs');

function readFilePromisified(filePath) {
  return new Promise((resolve, reject) => {
    fs.readFile(filename, 'utf8', (err, data) => {
      if (err) {
        reject(err);
      } else {
        resolve(data);
      }
    })
  });
}
```

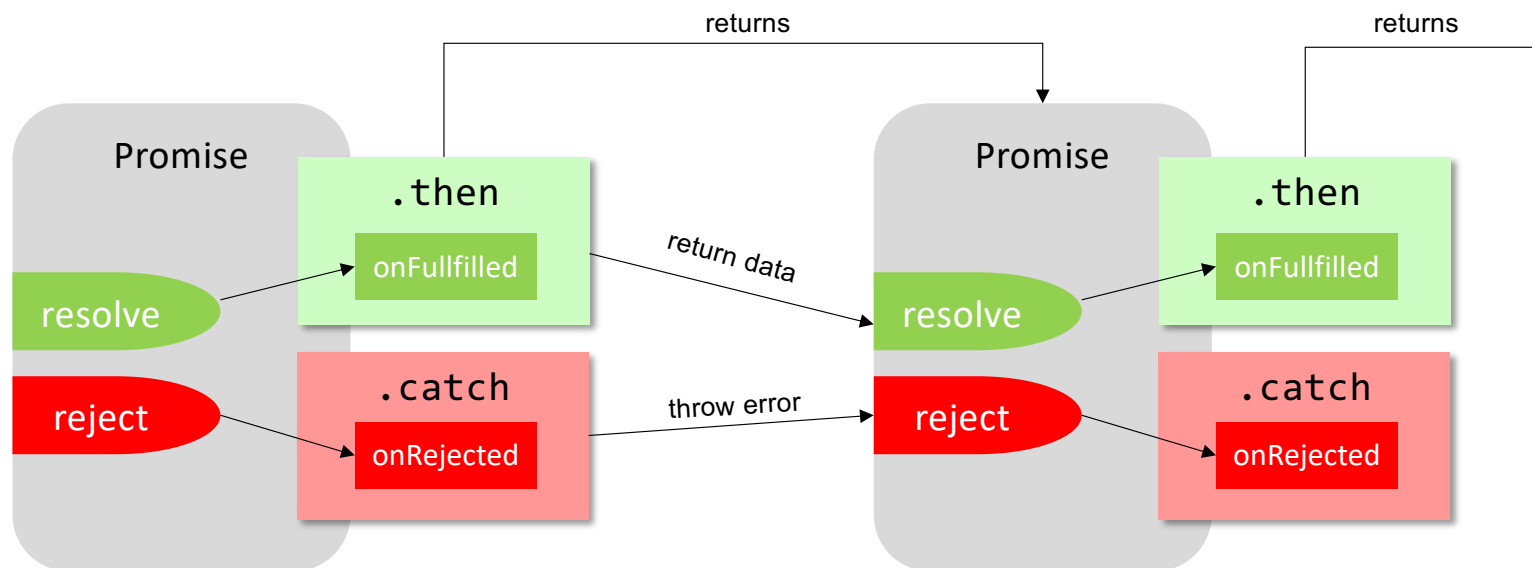


Wir werden noch sehen, dass gerade das return einer Promise vereinfacht wurde

Was machen eigentlich resolve und reject?

- `Promise.resolve(value)` und `Promise.reject(reason)` **liefern** tatsächlich einen **Promise**
- Formal ist also Chaining möglich
 - Beispiel: `fsProm(...).then(...).then(...).catch(...)`
 - Ein `catch()` reicht für alle rejects in der Kette!

Chaining: Sowohl then als auch catch geben neue Promise Objekte zurück



Syntax

Promises kaskadieren

```
readFilePromisified("bla.txt")  
.then(data => { console.log('CONTENT:', data)  
})  
.then( data => {  
    console.log("2nd then", data);  
})
```

Output:

```
CONTENT: This is the file contents!  
2nd then undefined
```

→ Chaining nur halb funktional, da Daten-Parameter nicht durchgereicht werden, console.log ist eben kein Promise!

Syntax

Promises kaskadieren

Lösung:

```
readFilePromisified("bla.txt")  
  .then(data => { console.log('CONTENT:', data)  
    return data  
  })  
  .then( data => {  
    console.log("2nd then", data);  
  })
```

→ Programmablauf einfacher nachzuvollziehen

Output:

```
1st then This is the file contents!  
2nd then This is the file contents!
```

→ **Hinweis:** Ein return in einem Promise führt zu einem resolve auf den Return-Wert

Promises verschönern die Callback-Hell

```
chooseToppings(function(toppings) {  
  placeOrder(toppings, function(order) {  
    collectOrder(order, function(pizza) {  
      eatPizza(pizza);  
    }, failureCallback);  
  }, failureCallback);  
}, failureCallback);
```

Alles was in einem Callback gemacht werden muss muss auch dort notiert werden. Asynchronität bedingt weitere Callbacks und keinen „normalen“ Ablauf

```
chooseToppings()  
  .then(toppings => placeOrder(toppings))  
  .then(order => collectOrder(order))  
  .then(pizza => eatPizza(pizza))  
  .catch(failureCallback);
```

```
chooseToppings()  
  .then(function(toppings) {  
    return placeOrder(toppings);  
  })  
  .then(function(order) {  
    return collectOrder(order);  
  })  
  .then(function(pizza) {  
    eatPizza(pizza);  
  })  
  .catch(failureCallback);
```

<https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Asynchronous/Promises>

Syntax

Asynchronität durch async / await

Verständlicher wird der Code durch das Deklarieren von asynchronen Funktionen und dem syntaktischen Anschein zu warten!

Beispiel:

```
async function read1st() {  
  data = await readFilePromisified("bla.txt");  
  console.log("1st then ", data);  
  return data; //wird zu Promise.resolve(data)  
}  
  
async function read2nd() {  
  data = await read1st();  
  console.log("2nd then ", data);  
}  
  
read2nd();
```

Output:

1st then This is the file contents!
2nd then This is the file contents!

Syntax

Asynchronität durch `async/await`

Was machen eigentlich `async` und `await`?

- `async` kann als Schlüsselwort vor dem Schlüsselwort `function` verwendet werden und **zeigt an, dass es asynchrone Abschnitte in der Funktion gibt**
- Eine `async` Funktion **liefert einen Promise!** Ein `return` wird auch hier automatisch in ein `Promise.resolve()` gewandelt
- **Innerhalb** einer asynchronen Funktion **kann `await` genutzt werden** um auf einen Promise zu "warten"
- `await` **durfte nur in `async` functions verwendet werden**, andernfalls `SyntaxError`. Ab `node.js` 14.8 gibt es **Top-Level `async` im ES-Modul-Modus**
- Um das `catch`-Verhalten eines Promises nachzubilden, muss ein `try/catch` benutzt werden

```
async function read1st() {  
  try{  
    data = await readFilePromisified("bla.txt");  
    console.log("1st then ", data);  
    return data; //wird zu Promise.resolve(data)  
  }  
  catch(e){ throw e } //wird zu Promise.reject(e)  
}
```

Syntax

Asynchronität durch `async/await`

Merke:

- `async` bedeutet nicht nur „es gibt asynchrone Abschnitte“, sondern formal: **Die Funktion wird so ausgeführt, dass ihr Rückgabewert immer ein Promise ist** und `await` ist darin erlaubt
- Eine `async`-Funktion liefert **immer** ein Promise
 - `Return x` entspricht `return Promise.resolve(x)`
 - `Throw e` entspricht `return Promise.reject(e)`
- `await` wartet nicht blockierend, sondern wertet den Ausgang des Promises aus und liefert den Code als Microtask über die Job-Queue aus, so dass dieser dann ausgeführt wird, wenn der Call-Stack abgearbeitet ist und der Code der erste Eintrag in der Queue ist

Achtung: Asynchrone Funktionen sollten trotzdem schnell sein

```
// -----  
// these functions simulate requests that need to run async  
// -----  
function asyncThing1() {  
  return new Promise( executor: (resolve) => {  
    setTimeout( handler: () => resolve( value: 'Thing 1 is done!'), timeout: 2000);  
  });  
}  
  
function asyncThing2() {  
  return new Promise( executor: (resolve) => {  
    setTimeout( handler: () => resolve( value: 'Thing 2 is done!'), timeout: 2000);  
  });  
}  
  
// -----  
// this is the code that actually loads data  
// -----  
function doThings() {  
  asyncThing1().then((thing1) => {  
    // do something with the first response  
    console.log(thing1);  
  });  
  
  asyncThing2().then((thing2) => {  
    // do something with the second response  
    console.log(thing2);  
  });  
}  
  
https://www.learnwithjason.dev/blog/keep-async-await-from-blocking-execution#promises-are-a-wonderful-powerful-tool
```

Achtung: Asynchrone Funktionen sollten trotzdem schnell sein

```
// ⚠ don't do this in your real code; it's slow!  
async function doThings() {  
  const thing1 = await asyncThing1();  
  console.log(thing1);  
  
  const thing2 = await asyncThing2();  
  console.log(thing2);  
}  
  
doThings();
```

<https://www.learnwithjason.dev/blog/keep-async-await-from-blocking-execution#promises-are-a-wonderful-powerful-tool>

Promise.all!

```
// ✅ do this - async code is run in parallel!  
async function doThings() {  
  const p1 = asyncThing1();  
  const p2 = asyncThing2();  
  
  const [thing1, thing2] = await Promise.all( values: [p1, p2]);  
  
  console.log(thing1);  
  console.log(thing2);  
}  
  
doThings();
```

<https://www.learnwithjason.dev/blog/keep-async-await-from-blocking-execution#promises-are-a-wonderful-powerful-tool>

await – Blockierend?

await konnte vor dem **ES-Modul-Modus** nur in Funktionen genutzt werden, die mit `async` ausgezeichnet sind! Moderne Browser können Top-Level await

`(async () => {...})` war schon immer möglich

`async`-Funktionen sind eigentlich Promises und der `return`-Wert ist ein automatisch fulfilled promise

await ist faktisch eine Schreibweise für ein `then()`.

Alles, was hinter await steht kommt in den then-Block

Damit wird der Thread nicht blockiert!

await ist syntaktischer Zucker für ein Chaining von promises. Anschaulich werden also callbacks auf die Job-Queue gelegt

await – Blockierend?

```
function foo() {  
  console.log("Hi, ");  
  return Promise.resolve(1);  
}
```

```
foo().then(result => console.log(result))  
console.log("Volker, ");
```



```
async function foo() {  
  console.log("Hi, ");  
  return 1;  
}
```

```
async function bar() {  
  const result = await foo();  
  console.log(result);  
}
```

```
bar();  
console.log("Volker, ");
```

Hi,
Volker,
1

[Call-Stack] → bar() → foo() → console.log('Hi, ')
↓
Promise.resolve(1) (fulfilled)
↓
await → *Microtask* (continue bar)
↓
[Call-Stack] → console.log('Volker, ') // noch
synchron
↓
(Call-Stack leer) → **Job-Queue** abarbeiten
↓
Microtask → result = 1 ; console.log(1)

await – Blockierend?

```
async function example() {  
  console.log('Start');  
  const result = await Promise.resolve(value: 'Hello');  
  console.log(result);  
  console.log('End');  
}
```



Alles was hier steht kommt
auf die Job-Queue

```
example();  
console.log('Ich komme vor dem Teil ab await');
```

Start

Ich komme vor dem Teil ab await

Hello

End

await – Blockierend?

```
function resolveAfter2Seconds() {  
  return new Promise(resolve => {  
    setTimeout(() => {  
      resolve('resolved');  
    }, 2000);  
  });  
}  
  
async function asyncCall() {  
  console.log('calling');  
  const result = await resolveAfter2Seconds();  
  console.log(result);  
  // expected output: "resolved"  
}  
  
(async() => { await asyncCall();  
              console.log("Wow");  
            })()  
  
console.log("Hui");
```

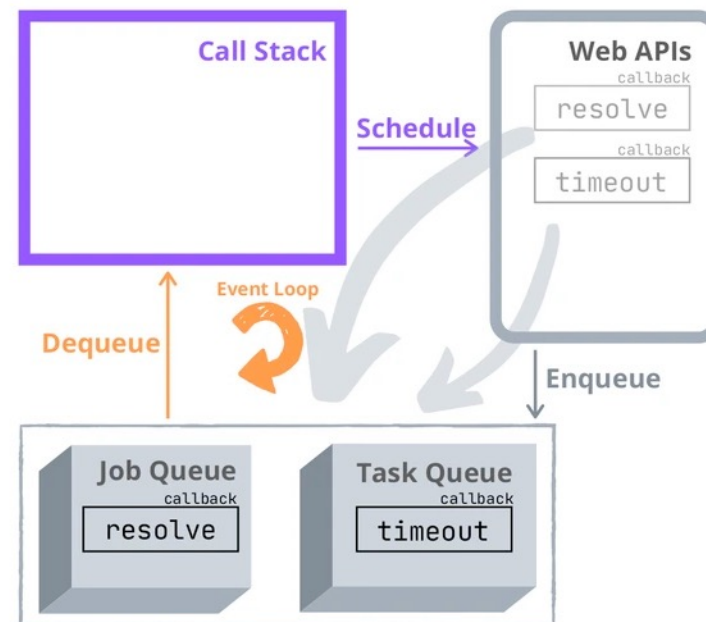
Liefert einen Promise!

```
> "calling"  
> "Hui"  
> "resolved"  
> "Wow"
```

Promises sind schneller als Timeouts!

Job Queue / Task Queue

```
setTimeout(function timeout() {  
  console.log('Timed out!');  
}, timeout: 0);  
  
Promise.resolve(value: 1).then(function resolve() {  
  console.log('Resolved!');  
});  
  
// logs 'Resolved!'  
// logs 'Timed out!'
```



<https://dmitripavlutin.com/javascript-promises-settimeout/>

Promises sind schneller als Timeouts!

```
async function foo() {  
  setTimeout(() => console.log('Timed out'), 0);  
  await Promise.resolve().then(() => console.log('Resolved'))  
  console.log('Wird ausgeführt')  
}
```

```
foo();  
console.log('Hello')
```

```
Hello  
Resolved  
Wird ausgeführt  
Timed out
```

Datenbankzugriff

Datenhaltung mittels Dateien nicht zu empfehlen

- Datenbanken!

Die meisten gängigen Datenbanken lassen sich mit Node.js nutzen

- (Oracle) MySQL
- MariaDB
- Oracle Database
- SQLite
- MS SQL
- PostgreSQL
- IBM Informix
- MongoDB (NoSQL)

Wir nutzen SQLite



Weitere Informationen (nicht wirklich...): <http://howfuckedismydatabase.com/>

Sequelize.js

- Third Party Node.js Modul für Datenbankzugriffe
- **ORM** (Datenbankzugriff ohne SQL-Queries schreiben zu müssen)
- Raw Queries auch möglich (nutzen wir auf Basis von Promises)
- Verbindungsaufbau:

```
const Sequelize = require('sequelize');

// Option 1: Passing parameters separately
const sequelize = new Sequelize('database', 'username', 'password', {
  host: 'localhost',
  dialect: /* one of 'mysql' | 'mariadb' | 'postgres' | 'mssql' */
});

// Option 2: Passing a connection URI
const sequelize = new
Sequelize('postgres://user:pass@example.com:5432/dbname');

// SQLite
const sequelize = new Sequelize({
  dialect: 'sqlite',
  storage: 'path/to/database.sqlite'
});
```

- Ein new sequelize liefert einen Promise

```
sequelize.authenticate()  
  .then(() => {  
    console.log('Connection has been established successfully.');  })  
  .catch((err) => {  
    console.error('Unable to connect to the database:', err);  
  });
```

Sequelize.js

- SQL-Abfrage mittels sequelize.query

```
sequelize.query("SELECT * FROM user WHERE id = " + id ,  
               {type: sequelize.QueryTypes.SELECT })  
  .then( userinfos => {  
    if(userinfos.length !== 0) {  
      console.log(userinfos[0].user)  
    }  
    else{  
      console.log("no users")  
    }  
  })
```

- anfällig für SQL Injections
- sollte für Abfragen nicht benutzt werden

Alternative: Prepared Statements

Exkurs: SQL Injection

- Fehlende Validierung/Maskierung von Benutzereingaben führt zu ungewollten Datenbankstatements
- Beispiel:

	Erwarteter Aufruf
Aufruf	<code>http://example.com/page?id=42</code>
Erzeugtes SQL	<code>SELECT titel, text FROM artikel WHERE ID=42;</code>

	Angriff mittels SQL-Injection
Aufruf	<code>http://example.com/page?id=42;DROP+TABLE+user</code>
Erzeugtes SQL	<code>SELECT titel, text FROM artikel WHERE ID=42;DROP TABLE user;</code>

Exkurs: SQL Injection

- Zweites Beispiel:

	Erwarteter Aufruf
Aufruf	<code>http://example.com/login (POST)</code> <code>user=admin&pw=honigkuchen</code>
Erzeugtes SQL	<code>SELECT id FROM user WHERE name = 'admin'</code> <code>AND pw = 'cee199b741247512eb298a34c1fa18f5ca704bae'</code>
	Angriff mittels SQL-Injection
Aufruf	<code>http://example.com/login</code> <code>user=admin'+OR+1='1'--&pw=asdf</code>
Erzeugtes SQL	<code>SELECT id FROM user WHERE name = 'admin' OR 1='1'</code> <code>--AND pw = '3da541559918a808c2402bba5012f6c60b27661c'</code>

Prepared Statements

Prepared Statements

- Vorbereitete Anweisung wird an die Datenbank gesendet
 - > Platzhalter für Variablen

- Ausführung des Statements nach Einsetzen der Variablen
 - > Datenbanksystem kümmert sich um das Escapen
 - > Verhindert SQL-Injections und prüft die Gültigkeit von Parametern

- Geschwindigkeitsvorteil bei mehrfachen Ausführen
 - > Statement liegt dem Datenbanksystem bereits vor

„Replacements“ (ggf. mit await davor)

- Ersetzung des Parameters per
 - „?“-Notation (**unnamed Parameter**)oder
 - „:“-Notation (named Parameter)

```
sequelize.query("SELECT * FROM user WHERE id = ?",  
               {replacements: [req.params.id], ,  
               type: sequelize.QueryTypes.SELECT })  
  .then( userinfos => {  
    if(userinfos.length !== 0){  
      console.log(userinfos["0"].user)  
    }  
    else{  
      console.log("no users")  
    }  
  })
```

„:“-Notation

```
sequelize.query("SELECT * FROM user WHERE id = :id" ,  
               {replacements: { id: req.params.id},  
               type: sequelize.QueryTypes.SELECT })  
    .then( userinfos => {  
        if(userinfos.length != 0){  
            console.log(userinfos["0"].user)  
        }  
        else{  
            console.log("no users")  
        }  
    })
```


Array-Replacements

```
sequelize.query("SELECT * FROM user WHERE id IN(:ids)" ,  
  { replacements: { ids: [1, 2, 3, 4]},  
    type: sequelize.QueryTypes.SELECT })  
  .then( userinfos => {  
  
    [...]  
  
  })
```

%-Operator

```
replacements: { search_name: 'an%' }
```

> Suche aller Namen, die mit ,an` beginnen

Sequelize.js

Gerne auch mit await for der Query
(in einer async-Funktion!)

```
const results = await sequelize.query("SELECT * FROM " +  
    "user WHERE id = ?" ,  
    {replacements: [req.params.id],  
     type: sequelize.QueryTypes.SELECT })  
console.log(results) // gibt die Zeilen aus
```

```
const [results, metadata] = await sequelize.query("...")
```

```
// metadata gibt bei einigen DB-Systemen zusätzliche
```

```
// Informationen an
```

Sequelize.js

ORM!!!

```
const Note = sequelize.define('notes',
  { note: Sequelize.TEXT, tag: Sequelize.STRING })

sequelize.sync({ force: true })
  .then(() => {
    console.log(`Database & tables created!`)
    Note.bulkCreate([
      { note: 'after work party', tag: 'fun' },
      { note: 'Scrum Meeting', tag: 'work' },
    ]).then(function() {
      return Note.findAll(); }).then(
        (notes) => console.log(notes))
  });

app.get('/notes', function(req, res) {
  Note.findAll().then(notes => res.json(notes)); });

app.get('/notes/:id', function(req, res) {
  Note.findAll({ where: { id: req.params.id }
  }).then(notes => res.json(notes)); });
```

JavaScript im Einsatz

Prototypen und ECMAScript

JS Exceptions

- Standardweg für Exception Handling
- Wenn ein Problem auftritt, wird eine Exception geworfen
- Klassische Statements: throw/try/catch
- **Erzeugung** über einen exception constructor (optionaler Übergabeparameter: message)
 - `const rangeException = new RangeError([message]);`
 - `const syntaxException = new SyntaxError([message]);`
- **Werfen** der Exception per throw:
 - `throw rangeException;`

JS Exceptions

- Gängige Exception-Typen in Javascript
 - Error – genereller Fehler (Basis-Klasse)
 - ReferenceError – nicht valide Referenz
 - TypeError – nicht valider Typ
 - SyntaxError – falsche Syntax von Quellcode
 - RangeError – Parameter außerhalb valider Grenzen
- Eigenschaften der Error-Klasse:
 - name, message (Beschreibung der Fehlermeldung), stack (stack trace)

JS Exceptions

- Exception Handling
 1. Fangen der Exception
 - try-catch
 2. Fehlerbehebung
 - Anzeigen der Fehlermeldung; Wiederholen, Default-Aktion ausführen
 3. Ausführung der Applikation fortsetzen
 - Applikation wird in der ersten Zeile nach dem catch-Block fortgeführt

```
try {  
    // code that can throw an exception  
} catch (ex) {  
    // this code is executed in case of exception  
    // and ex holds the info about the exception  
}
```

Try-Blöcke werden oftmals nicht von der JS-Engine optimiert, so dass hier
Besser keine durchsatzkritischen Maßnahmen getroffen werden sollten

JS Exceptions

Handlen von mehreren unterschiedlichen Exceptions

- **Jedes try-catch kann nur einen catch-Block enthalten**
- Filtern des Exception-Types wie in Java oder C++ nicht möglich (multiple Catches – abhängige catch-Abschnitte sind nicht Teil der Standardisierung und sollten nicht verwendet werden)
- Die ECMAScript-Spezifikation erlaubt nur folgende Syntax:

```
try {  
    // Some code causing different types of exceptions  
} catch (ex) {  
    if (ex instanceof TypeError) { ... }  
    else if (ex instanceof ReferenceError) { ... }  
    else if (ex instanceof SyntaxError) { ... }  
}
```


Vererbung in JavaScript ist anders

Konstante können geändert werden

```
const vs = {name: "Volker Sander", beruf: "Professor"}
```

```
vs.name = "V.Sander"
```

```
console.log(vs)
```

Vererbung in JavaScript ist anders

Prototypen

Klassenlose Vererbung in JavaScript

- Über Prototypen (Objekte erben von Objekten)
 - > Jedes Objekt (und damit jede Funktion) besitzt in JavaScript ein privates Attribut welches auf das sogenannten Prototypobjekt zeigt
 - > Auch das Prototypobjekt besitzt ein solches, bis auf die Mutter aller Objekte, das Object-Objekt. Hier ist das Attribut null
 - > Ein Prototypobjekt kann Attribute und Methoden(Funktionen) haben
 - > Vererbung wird anschaulich auf Basis der Verlinkung der Prototypobjekte realisiert

Basis für unsere Klassen/Objekte sind zunächst Funktionen

Funktionen können über die Prototypen Attribute erhalten

Objekte können damit Attribute und Funktionen vom Prototypobjekt „erben“

Klassenlose Vererbung in JavaScript

- Über Prototypen (Objekte erben von Objekten)
 - > **Jede Funktion hat ein prototype-Attribut.** Sinnig sind diese eigentlich bei Verwendung von Konstruktoren mittels „new“. Alle Objekte, die mittels des Konstruktors erzeugt wurden haben das gleiche prototype-Attribut
 - > Im **Prototypobjekt können jederzeit Attribute und Methoden gespeichert werden**, auf die auch neu erzeugte Objekte zugreifen können
 - > Auch Prototypen haben einen Prototypen, wobei das Object-Objekt keinen Prototypen hat (null)
 - Es ergibt sich eine „Prototype-Chain“
 - > Mit der Verlinkung von Prototypen können so Vererbungsmechanismen erzeugt werden

Prototypen

Basis für Vererbung

```
}function Person(name) {  
    this.name = name;  
}  
Person.prototype.greet = function() {  
    console.log(`Hello, my name is ${this.name}`);  
}  
  
const john = new Person( name: 'John');  
john.greet(); // Outputs "Hello, my name is John"  
  
Person.prototype.age = 30;  
  
const jane = new Person( name: 'Jane');  
jane.greet(); // Outputs "Hello, my name is Jane"  
console.log(jane.age); // Outputs 30  
console.log(john.age); // Outputs 30
```

Prototypen

Zugriff auf die Prototypen

Prototype und __proto__

- **[Funktion].prototype**

- > „Magisches“ Attribut prototype, das jede Funktion besitzt
- > Grundlage für die Vererbung. Wird benutzt um das interne __proto__ für alle zu setzen, mittels des gleichen Konstruktors erzeugt wurden
- > Enthält gemeinsam nutzbare Methoden und Attribute, auf die die neu erzeugten Objekte zugreifen können

- **[Objekt].__proto__**

- > „Magisches“ **internes** Attribut __proto__, das jedes Objekt besitzt
- > Referenziert die Eigenschaft prototype des Objekts (nach Erzeugung mittels new)

Zugriff auf ein Attribut – Ablauf

- Beispiel: `alert(obj.a)` ;

- > Besitzt obj ein Attribut a? Falls ja, nehmen wir den Wert
 - Sonst: Besitzt der obj.__proto__ ein Attribut a? Falls ja, nehmen wir den Wert
 - Sonst: Besitzt obj.__proto__.__proto__ ein ...
- > Solange durchführen bis ein Attribut a gefunden ist (sonst undefined)

Weitere Informationen: https://developer.mozilla.org/de/docs/Web/JavaScript/Inheritance_and_the_prototype_chain

```
function f() {}  
f.prototype.b = 123;  
  
const obj = new f();  
console.log(obj.b); // Output: 123  
  
console.log(obj.__proto__ === f.prototype); // Output: true  
  
console.log(f.__proto__ === Function.prototype); // Output: true  
  
console.log(obj.__proto__.b); // Output: 123
```

```
let f = function () {  
    this.a = 1;  
    this.b = 2;  
}  
  
let o = new f(); // {a: 1, b: 2}  
f.prototype.summe = function () {  
    return this.a + this.b;  
};  
  
console.log(o.summe()); // 3
```

```
let f = function () {  
    // .this ordnet die Attribute dem aufrufenden Objekt zu  
    this.a = 1;  
    this.b = 2;  
}  
  
let o = new f(); // {a: 1, b: 2}  
f.prototype.b = 3;  
f.prototype.c = 4;  
console.log(o.a); // 1  
console.log(o.b); // 2 und nicht etwa 3! Suchhierarchie!!!  
console.log(o.__proto__.b) // 3  
console.log(o.c); // 4  
console.log(o.d); // undefined  
f.e = 5; // ist jetzt ein Attribut der Funktion f und nicht des Objekts!  
console.log(f.e); // 5  
console.log(o.e); // undefined
```


Anwendung bei der Vererbung

```
function Fahrzeug(speed) {  
    this.speed = speed; // Eigenschaften werden an das  
    this.distance = 0; // Objekt selber gebunden  
}  
  
// Funktion für Objekte des Typs Fahrzeug  
Fahrzeug.prototype.go = function(times) {  
    this.distance += this.speed * times;  
};  
  
// porsche.__proto__ === Fahrzeug.prototype (auch für opel)  
var porsche = new Fahrzeug( speed: 260);  
var opel     = new Fahrzeug( speed: 140);  
opel.go( times: 2);  
  
console.log(opel.distance); // 280
```

Anwendung bei der Vererbung

```
function Fahrzeug(speed) {  
    this.speed = speed;  
    this.distance = 0;  
};  
Fahrzeug.prototype.go = function(times) {  
    this.distance += this.speed * times;  
};  
const fahrzeug1 = new Fahrzeug( speed: 120);  
fahrzeug1.go( times: 2);  
function Auto(speed, fabrikat) {  
    Fahrzeug.call(this, speed);  
    this.fabrikat = fabrikat;  
}  
// Wir erzeugen ein Objekt, dass die prototype-Funktionen von Fahrzeug enthält  
Auto.prototype = new Fahrzeug();  
Auto.prototype.hupen = function() { // Wir hängen eine weitere Funktion an  
    console.log('Möööp!');  
}  
const auto = new Auto( speed: 150, fabrikat: 'Opel');  
auto.hupen();           // Gibt aus: 'Möööp!'  
auto.go( times: 1);  
console.log(auto.distance); // 150
```

Anwendung bei der Vererbung

Object.create

- Einfache Möglichkeit Prototyping zu nutzen
- Beispiel 1

```
const foo = {  
  eins : 1,  
  zwei : 2,  
  pi : 3.14159  
};
```

```
let bar = Object.create(foo); // Objekt mit foo als Prototyp  
bar.drei = 3;
```

```
// Eigenschaften von bar:  
console.log(bar.eins); // 1 (vom Prototyp geerbt)  
console.log(bar.zwei); // 2 (vom Prototyp geerbt)  
console.log(bar.drei); // 3  
console.log(bar.pi); // 3.14159
```

Object.create

- Einfache Möglichkeit Prototyping zu nutzen
- Beispiel 2

```
const foo = {  
  eins : 1,  
  zwei : 2,  
  pi : 3.14159  
};
```

```
let bar = Object.create(foo); // Objekt mit foo als Prototyp  
bar.drei = 3;  
bar.pi = 4;  
// Eigenschaften von bar:  
console.log(bar.eins); // 1 (vom Prototyp geerbt)  
console.log(bar.zwei); // 2 (vom Prototyp geerbt)  
console.log(bar.drei); // 3  
console.log(bar.pi); // 4 (Eigenschaft vom Prototypen überdeckt)
```

Object.create: Beispiel 2

```
var obj1 = {  
    a : 1  
};  
var obj2 = Object.create(obj1);  
obj2.b = 2;  
var obj3 = Object.create(obj2);  
obj3.c = 3;
```

```
obj1;    // {a: 1 }  
obj2;    // {a: 1, b: 2 }  
obj3;    // {a: 1, b: 2, c: 3 }
```

Object.create

```
Object.create(proto [, propertiesObject])
```

- Erstellt ein Objekt mit *proto* als `__proto__`
 - > Wird *null* übergeben, so wird ein Objekt ohne Prototyp erstellt
- Per *propertiesObject* können Eigenschaften des Objektes, sowie Getter- und Setter-Funktionen bestimmt werden
 - > Hier zunächst nicht weiter berücksichtigt

Welche Möglichkeiten kennen wir nun Objekte zu erstellen?

- Object.create

```
var obj = Object.create(null);  
obj.foo = 1;  
obj.bar = 2;
```

- JSON-like-Notation

```
var obj = {  
    foo : 1,  
    bar : 2  
};
```

- Konstrukturfunktion

```
var obj = new Object();  
obj.foo = 1;  
obj.bar = 2;
```

ECMAScript 7

Veröffentlicht Juni 2016

- Viel geplant, am Ende wenig Neues
 - > Math.pow-Kurzschreibweise: $x ** y$
 - > Array.prototype.includes
 - > **Async await als wesentliche Neuerung**
- Für die nächste Version mehr Features geplant

→ **guter**

Browser Support

(Stand April 2023)

ECMAScript 7

		99%	Compilers/polyfills				1%	98%	100%	99%	Desktop browsers						92%	96%	96%	99%	Server				
			74%	50%	51%	5%														79%	97%	99%	99%	99%	
Feature name	Current browser	Babel 7 + core-js 3	Closure 2022.07	Type-Script + core-js 3	es7-shim	IE 11	FF 102 ESR	FF 112	CH 110	Edge 110	SF 15	SF 15.2	SF 15.4	SF 16	OP 95	Node >=14.6 <15 ^[2]	Node >=16.11 <17	Node >=18.0 <18.3	Node >=18.3 <19	Node >=18.3 <19	Node >=19				
2016 features																									
● exponentiation (**) operator	3/3	3/3	3/3	2/3	0/3	0/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3	3/3				
● Array.prototype.includes	4/4	4/4	3/4	4/4	3/4	0/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4				
2016 misc																									
● generator functions can't be used with "new"	Yes	?	?	?	?	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes				
● generator throw() caught by inner generator	Yes	Yes	Yes	Yes ^[9]	?	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes				
● strict fn w/ non-strict non-simple params is error	Yes	Yes	?	?	?	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes				
● nested rest destructuring declarations	Yes	Yes	Yes	Yes	?	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes				
● nested rest destructuring parameters	Yes	Yes	Yes	Yes	?	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes				
● Proxy, "enumerate" handler removed	Yes	?	?	?	?	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes				
● Proxy internal calls, Array.prototype.includes	Yes	?	?	?	?	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes				
2017 features																									
● Object static methods	4/4	4/4	3/4	4/4	3/4	0/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4				
● String padding	2/2	2/2	2/2	2/2	2/2	0/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2				
● trailing commas in function syntax	2/2	2/2	2/2	2/2	0/2	0/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2				
● async functions	16/16	12/16	12/16	9/16	0/16	0/16	16/16	16/16	16/16	16/16	16/16	16/16	16/16	16/16	16/16	16/16	16/16	16/16	16/16	16/16	16/16				
● shared memory and atomics	12/17	0/17	0/17	0/17	0/17	0/17	17/17	17/17	17/17	17/17	0/17	17/17	17/17	17/17	17/17	17/17	17/17	17/17	17/17	17/17	17/17				

Quelle: <http://kangax.github.io/compat-table/es2016plus/>

Transpilation: Babel

Babel is a JavaScript compiler.



BABEL JS

<https://babeljs.io/>

- It turns ES2015:

```
const adding = (a, b) => a + b
```

- into old JavaScript:

```
'use strict';  
var adding = function adding(a, b) {  
  return a + b;  
};
```

TypeScript Compiler (tsc) wird beispielsweise standardmäßig bei Angular verwendet, es geht aber auch Babel

Bei der Nutzung von node.js sollten Sie

- Event Loop nicht blockieren
- Fehler immer sauber behandeln
- Async-Code konsequent und einheitlich schreiben
- Streaming für große Datenmengen nutzen
- Projekt sauber strukturieren
- Skalierung und Ressourcenverbrauch früh mitdenken